Sistema de Administração de um Reino Medieval

Documentação dos endpoints da API REST e das regras de negócio

Repositório	https://github.com/yeFF9/Reino-Medieval
Tecnologias	Node.js 20+, Express 5, PostgreSQL, Prisma, JWT, Zod, Swagger
Base da API	`http://localhost:3000/api/v1`
Total de endpoints	65 (além de `GET /health` e da Swagger UI em `/api-docs`)
Contribuidores	yeFF9, vinises, Matheus Morelo, Rafael Brandão

1. Convenções gerais

Autenticação e autorização

- Exceto `/auth/register`, `/auth/login` e `/health`, todas as rotas exigem o cabeçalho `Authorization: Bearer <JWT>`. Sem token, token inválido/expirado ou usuário inativo: **401**.
- O acesso é controlado por papéis (RBAC). Cada rota exige uma permissão (ex.: `reinos:write`); se nenhum papel do usuário a possuir, retorna **403**.
- Usuário de demonstração criado pelo seed: `admin@eldoria.test` / `Eldoria123!` (papel ADMIN).

Papéis definidos no seed

Papel	Permissões
ADMIN / MONARCA	Todas as permissões cadastradas.
ESTRATEGISTA	reinos:read, territorios:read, exercitos:*, guerras:*, comandantes:*, guardas:*
DIPLOMATA	reinos:read, territorios:read, guerras:read, diplomacia:*, tributos:*, economia:read
LEITOR	Leitura de reinos, territórios, cidades, exércitos, guerras, comandantes, guardas, diplomacia, tributos, guildas e economia.

Validação, paginação e respostas

- Entradas (body, params e query) são validadas com Zod. Identificadores devem ser UUID. Falhas retornam **400** no formato `{ "error": "Dados inválidos", "details": [{ "field", "message" }] }`.
- Listagens aceitam `page` (padrão 1) e `limit` (padrão 20, máximo 100) e respondem `{ data: [...], pagination: { page, limit, total, pages } }`.
- Criações e consultas individuais respondem `{ data: {...} }`. Erros de negócio respondem `{ "error": "mensagem" }`.

Código	Significado
200 / 201	Sucesso / recurso criado
400	Dados inválidos (validação) ou recurso insuficiente em operações de castelo e suprimentos
401 / 403	Não autenticado / sem permissão
404	Recurso referenciado não encontrado
409	Conflito de estado (duplicidade, status já alterado, concorrência em transação)
422	Regra de negócio violada
500	Erro interno do servidor

2. Resumo dos endpoints

Módulo	Endpoints
Autenticação	2
Reinos e Economia	5
Territórios	3

Cidades, População, Agricultura e Suprimentos	10
Exércitos	5
Guerras	3
Comandantes	4
Guardas e Patrulhas	4
Castelos	4
Diplomacia	4
Tributos	4
Guildas, Encomendas e Membros	12
Leis e Impostos	5

Método	Rota (prefixo /api/v1)	Permissão
POST	`/auth/register`	Pública
POST	`/auth/login`	Pública
GET	`/reinos`	reinos:read
POST	`/reinos`	reinos:write
GET	`/reinos/:id`	reinos:read
GET	`/reinos/:id/economia`	economia:read
PATCH	`/reinos/:id/economia`	economia:write
GET	`/territorios`	territorios:read
POST	`/territorios`	territorios:write
PUT	`/territorios/:id`	territorios:write
GET	`/cidades`	cidades:read
POST	`/cidades`	cidades:write
GET	`/cidades/:id/populacao`	cidades:read
GET	`/cidades/:id/populacao/mapa-fome`	cidades:read
GET	`/cidades/:id/populacao/satisfacao`	cidades:read
GET	`/cidades/:id/agricultura`	cidades:read
POST	`/agricultura/colheitas`	cidades:write
GET	`/cidades/:id/suprimentos`	cidades:read
GET	`/cidades/:id/racoes`	cidades:read
PUT	`/cidades/:id/suprimentos/converter`	cidades:write
GET	`/exercitos`	exercitos:read
POST	`/exercitos`	exercitos:write
PATCH	`/exercitos/:id/mobilizar`	exercitos:write
PATCH	`/exercitos/:id/desmobilizar`	exercitos:write
PATCH	`/exercitos/:id/designar-comandante`	exercitos:write

POST	`/guerras/declarar`	guerras:write
GET	`/guerras/historico`	guerras:read
PATCH	`/guerras/:id/finalizar`	guerras:write
GET	`/comandantes`	comandantes:read
POST	`/comandantes`	comandantes:write
GET	`/comandantes/:id`	comandantes:read
PATCH	`/comandantes/:id/promover`	comandantes:write
POST	`/guardas/patrulhas`	guardas:write
GET	`/patrulhas`	guardas:read
PATCH	`/patrulhas/:id/reforcar`	guardas:write
PATCH	`/patrulhas/:id/encerrar`	guardas:write
POST	`/castelos`	castelos:write
GET	`/castelos/:id`	castelos:read
PATCH	`/castelos/:id/melhorar`	castelos:write
POST	`/castelos/:id/reparar`	castelos:write
POST	`/diplomacia/pactos`	diplomacia:write
GET	`/diplomacia/pactos/:reinoId`	diplomacia:read
DELETE	`/diplomacia/pactos/:id/romper`	diplomacia:write
POST	`/diplomacia/tributos/enviar`	diplomacia:write
GET	`/tributos`	tributos:read
GET	`/tributos/:id`	tributos:read
GET	`/tributos/balanco/:reinoId`	tributos:read
PATCH	`/tributos/:id/estornar`	tributos:write
POST	`/guildas`	guildas:write
GET	`/guildas/:id`	guildas:read
GET	`/guildas/:id/membros`	guildas:read
POST	`/guildas/:id/encomendas`	guildas:write
GET	`/encomendas`	guildas:read
GET	`/encomendas/:id`	guildas:read
PATCH	`/encomendas/:id/concluir`	guildas:write
PATCH	`/encomendas/:id/cancelar`	guildas:write
GET	`/membros-guilda`	guildas:read
POST	`/membros-guilda`	guildas:write
PATCH	`/membros-guilda/:id/transferir`	guildas:write
DELETE	`/membros-guilda/:id`	guildas:write
GET	`/leis`	leis:read

POST	`/leis`	leis:write
PUT	`/leis/:id/revogar`	leis:write
POST	`/leis/impostos/aplicar`	impostos:write
GET	`/leis/impostos/historico`	impostos:read

3. Detalhamento dos endpoints

Parâmetros marcados com `?` são opcionais. Os valores padrão entre parênteses podem ser alterados pelas variáveis de ambiente indicadas.

Autenticação

Rotas públicas para criar conta e obter o token JWT usado em todas as demais rotas.

1 POST `/api/v1/auth/register` — Pública

Registra um novo usuário.

ENTRADA

- Body: `nome` (2–100), `email` (e-mail válido, até 254), `senha` (8–128).

REGRAS DE NEGÓCIO

- O e-mail deve ser único; duplicado retorna **409**.
- Todo usuário registrado recebe o papel **LEITOR**.
- A senha é armazenada como hash bcrypt (custo 12), nunca em texto puro.

SUCESSO

201 — dados do usuário (id, nome, email, createdAt).

2 POST `/api/v1/auth/login` — Pública

Autentica o usuário e devolve o token.

ENTRADA

- Body: `email`, `senha` (8–128).

REGRAS DE NEGÓCIO

- Usuário inexistente, inativo ou senha incorreta retornam **401** com a mesma mensagem (não revela qual falhou).
- O JWT carrega o id do usuário (`sub`) e expira em `JWT_EXPIRES_IN` (padrão 1h).

SUCESSO

200 — `{ token, tokenType: "Bearer", expiresIn }`.

Reinos e Economia

3 GET `/api/v1/reinos` — reinos:read

Lista reinos.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `nome`, `formaGoverno` (busca parcial, sem diferenciar maiúsculas).

SUCESSO

200 — lista paginada.

4 **POST** **`/api/v1/reinos`** reinos:write

Cria um reino.

ENTRADA

- Body: `nome` (2–100), `descricao`? (até 500), `formaGoverno`? (até 80).

REGRAS DE NEGÓCIO

- O nome do reino é único (**409** se repetido).
- Todo reino nasce com uma economia zerada (ouro, alimentos e ferro = 0).

SUCESSO

201 — reino com economia.

5 **GET** **`/api/v1/reinos/:id`** reinos:read

Detalha um reino, com economia e territórios.

REGRAS DE NEGÓCIO

- Reino inexistente: **404**.

SUCESSO

200.

6 **GET** **`/api/v1/reinos/:id/economia`** economia:read

Consulta os recursos do reino.

REGRAS DE NEGÓCIO

- **404** se o reino não existir ou não tiver economia registrada.

SUCESSO

200 — ouro, alimentos, ferro.

7 **PATCH** **`/api/v1/reinos/:id/economia`** economia:write

Ajusta os recursos do reino.

ENTRADA

- Body: `modo` (`CREDITAR` | `DEFINIR`, padrão CREDITAR), `ouro`?, `alimentos`?, `ferro`? (≥ 0, ao menos um).

REGRAS DE NEGÓCIO

- **CREDITAR** soma os valores ao saldo atual; **DEFINIR** substitui o saldo.
- Se o reino ainda não tiver economia, ela é criada.

SUCESSO

200 — economia atualizada.

Territórios

8 **GET** **`/api/v1/territorios`** territorios:read

Lista territórios.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `nome`, `status`, `reinoId`.

SUCESSO

200 — lista paginada com o reino dono.

9 **POST** `/api/v1/territorios` territorios:write

Cria um território.

ENTRADA

- Body: `nome` (2–100), `descricao`?, `status`?, `reinoId`? (pode ser nulo).

REGRAS DE NEGÓCIO

- Cada território tem no máximo um reino dono (`reinoId`); o reino precisa existir (**404**).
- Nome único (**409**).

SUCESSO

201.

10 **PUT** `/api/v1/territorios/:id` territorios:write

Atualiza um território.

ENTRADA

- Body: qualquer combinação de `nome`, `descricao`, `status`, `reinoId` (ao menos um campo).

REGRAS DE NEGÓCIO

- Território ou novo reino inexistente: **404**.
- Nome repetido: **409**.

SUCESSO

200.

Cidades, População, Agricultura e Suprimentos

11 **GET** `/api/v1/cidades` cidades:read

Lista cidades.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `nome`, `reinoId` (via território), `populacaoMinima`.

SUCESSO

200.

12 **POST** `/api/v1/cidades` cidades:write

Funda uma cidade.

ENTRADA

- Body: `nome`, `territorioId`, `populacao` (inteiro > 0), `estoqueAlimentos` (≥ 0).

REGRAS DE NEGÓCIO

- O estoque de alimentos inicial deve ser ≥ `MIN_CITY_FOOD_STOCK` (padrão 100), senão **422**.
- O território precisa existir (**404**).
- A produção agrícola inicial é calculada: `populacao × 0,01`.
- Nome de cidade único (**409**).

SUCESSO

201.

13 GET `/api/v1/cidades/:id/populacao`

cidades:read

Censo da cidade.

SUCESSO

200 — cidadeId, nome, território e população.

14 GET `/api/v1/cidades/:id/populacao/mapa-fome`

cidades:read

Avalia a segurança alimentar.

REGRAS DE NEGÓCIO

- Necessidade diária = `populacao × 2`; média de refeições = `estoque ÷ necessidade × 2`.
- Classificação: < 0,5 *Grave Ameaça de Fome*; < 1,0 *Racionamento Severo*; < 1,5 *Abastecimento Adequado*; ≥ 1,5 *Fartura*.
- `atendeReservaMinima` é verdadeiro quando o estoque cobre a necessidade diária.

SUCESSO

200.

15 GET `/api/v1/cidades/:id/populacao/satisfacao`

cidades:read

Índice de satisfação (0–10) e risco de revolta.

REGRAS DE NEGÓCIO

- Média de refeições ≥ 1,5 → 10/10, 0% de revolta (*Povo Leal & Próspero*).
- ≥ 1,0 → 7/10, 5% (*Povo Satisfeito*).
- ≥ 0,5 → 3/10, 45% (*Tensão Social / Risco Alto*).
- < 0,5 → 0/10, 90% (*Revolta Iminente*).

SUCESSO

200.

16 GET `/api/v1/cidades/:id/agricultura`

cidades:read

Produção agrícola e estoque atual.

SUCESSO

200.

17 POST `/api/v1/agricultura/colheitas`

cidades:write

Registra uma colheita.

ENTRADA

- Body: `cidadeId`, `quantidadeBruta` (≥ 1).

REGRAS DE NEGÓCIO

- A quantidade colhida passa a ser a produção agrícola atual e é somada ao estoque de alimentos.
- Cidade inexistente: **404**.

SUCESSO

201 — cidade atualizada.

18 **GET** `/api/v1/cidades/:id/suprimentos` cidades:read

Situação do estoque de alimentos.

REGRAS DE NEGÓCIO

- Reserva mínima exigida = `populacao × 2`; informa se a cidade a atende.

SUCESSO

200.

19 **GET** `/api/v1/cidades/:id/racoes` cidades:read

Estoque de rações militares.

SUCESSO

200.

20 **PUT** `/api/v1/cidades/:id/suprimentos/converter` cidades:write

Converte alimentos em rações militares.

ENTRADA

- Body: `percentualRacao` (0–100).

REGRAS DE NEGÓCIO

- Quantidade convertida = `producaoAgricola × percentual ÷ 100`.
- O estoque de alimentos precisa cobrir a quantidade, senão **400**.
- O valor sai do estoque de alimentos e entra no estoque de rações.

SUCESSO

200.

Exércitos

21 **GET** `/api/v1/exercitos` exercitos:read

Lista exércitos com reino e comandante.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `nome`, `reinoId`, `status` (DISPONIVEL | MOBILIZADO).

SUCESSO

200.

22 **POST** `/api/v1/exercitos` exercitos:write

Cria um exército.

ENTRADA

- Body: `nome`, `reinoId`, `territorioId`, `comandanteId`, `efetivo` (inteiro > 0).

REGRAS DE NEGÓCIO

- Reino, território e comandante precisam existir (**404**).
- O exército só pode ser estacionado em território do próprio reino (**422**).
- O comandante precisa ter patente nível ≥ 3 (**422**).

SUCESSO

201.

23 **PATCH** **`/api/v1/exercitos/:id/mobilizar`** exercitos:write

Mobiliza o exército.

REGRAS DE NEGÓCIO

- Exército já mobilizado: **409**.
- O reino precisa ter no mínimo 100 de ouro, 50 de alimentos e 25 de ferro (`MIN_ARMY_*`), senão **422**; esses valores são debitados.
- Custo de manutenção = `efetivo × 0,05` (`MAINTENANCE_PER_SOLDIER`).
- Débito e mudança de status ocorrem em transação serializável; conflito concorrente retorna **409**.

SUCESSO

200.

24 **PATCH** **`/api/v1/exercitos/:id/desmobilizar`** exercitos:write

Desmobiliza o exército.

REGRAS DE NEGÓCIO

- Exército já disponível: **409**.
- Volta ao status DISPONIVEL e o custo de manutenção é zerado.

SUCESSO

200.

25 **PATCH** **`/api/v1/exercitos/:id/designar-comandante`** exercitos:write

Troca o comandante do exército.

ENTRADA

- Body: `comandanteId`.

REGRAS DE NEGÓCIO

- Exército ou comandante inexistente: **404**.
- Mesma regra de patente mínima (nível 3) da criação (**422**).

SUCESSO

200.

Guerras

26 **POST** **`/api/v1/guerras/declarar`** guerras:write

Declara guerra por um território.

ENTRADA

- Body: `reinoAtacanteId`, `reinoDefensorId` (diferentes), `territorioId`.

REGRAS DE NEGÓCIO

- Um reino não pode declarar guerra contra si mesmo (**400**).
- O território disputado deve pertencer ao reino defensor (**422**).
- Não é permitido atacar um reino com **aliança** ou **pacto de não agressão** ativo (**409**).
- A guerra começa com status EM_ANDAMENTO.

SUCESSO

201.

27 **GET** `/api/v1/guerras/historico` guerras:read

Histórico de guerras, mais recentes primeiro.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `reinoId` (como atacante ou defensor), `status` (EM_ANDAMENTO | FINALIZADA).

SUCESSO

200.

28 **PATCH** `/api/v1/guerras/:id/finalizar` guerras:write

Encerra a guerra e define o vencedor.

ENTRADA

- Body: `vencedorId`.

REGRAS DE NEGÓCIO

- Guerra já finalizada: **409**.
- O vencedor deve ser um dos dois reinos em guerra (**422**).
- Na mesma transação, o território disputado passa ao vencedor com status CONTROLADO.

SUCESSO

200.

Comandantes

29 **GET** `/api/v1/comandantes` comandantes:read

Lista comandantes.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `nome`, `patente`, `nivelPatenteMinimo`.

SUCESSO

200.

30 **POST** `/api/v1/comandantes` comandantes:write

Cadastra um comandante.

ENTRADA

- Body: `nome` (2–100), `patente` (2–80), `nivelPatente` (1–10).

SUCESSO

201.

31 **GET** `/api/v1/comandantes/:id` comandantes:read

Detalha o comandante e seus exércitos.

REGRAS DE NEGÓCIO

- Inexistente: **404**.

SUCESSO

200.

32 **PATCH** **`/api/v1/comandantes/:id/promover`** comandantes:write

Promove o comandante.

ENTRADA

- Body: `nivelPatente` (1–10), `patente`? (novo título).

REGRAS DE NEGÓCIO

- O novo nível deve ser maior que o atual (**422**).

SUCESSO

200.

Guardas e Patrulhas

33 **POST** **`/api/v1/guardas/patrulhas`** guardas:write

Envia uma patrulha para a cidade.

ENTRADA

- Body: `cidadeId`, `contingente` (inteiro > 0).

REGRAS DE NEGÓCIO

- A cidade precisa de um castelo no seu território (**422**).
- O contingente não pode exceder os guardas disponíveis no castelo (**422**).
- Os guardas saem do castelo em transação serializável; a patrulha nasce ATIVA.

SUCESSO

201.

34 **GET** **`/api/v1/patrulhas`** guardas:read

Lista patrulhas.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `status` (ATIVA | ENCERRADA), `cidadeId`, `contingenteMinimo`.

SUCESSO

200.

35 **PATCH** **`/api/v1/patrulhas/:id/reforcar`** guardas:write

Adiciona guardas a uma patrulha.

ENTRADA

- Body: `contingente` (inteiro > 0).

REGRAS DE NEGÓCIO

- Somente patrulhas ATIVA podem ser reforçadas (**409**).
- Limite: 1 guarda para cada 100 habitantes (`CITIZENS_PER_GUARD`) (**422**).
- Os guardas são retirados do castelo, que precisa tê-los disponíveis (**422**).

SUCESSO

200.

36 **PATCH** **`/api/v1/patrulhas/:id/encerrar`** guardas:write

Encerra a patrulha.

REGRAS DE NEGÓCIO

- Patrulha já encerrada: **409**.
- Os guardas voltam ao castelo até a capacidade estrutural; o excedente é dispensado.

SUCESSO

200 — inclui `guardasRetornados` e `guardasDispensados`.

Castelos

37 **POST** **`/api/v1/castelos`** castelos:write

Constrói um castelo em um território.

ENTRADA

- Body: `territorioId`, `capacidadeGuardas` (inteiro ≥ 1).

REGRAS DE NEGÓCIO

- Território inexistente: **404**.
- Cada território comporta um único castelo (**409**).
- O castelo nasce com muralha nível 1 e integridade 100%.

SUCESSO

201.

38 **GET** **`/api/v1/castelos/:id`** castelos:read

Detalha o castelo, seu território e reino.

SUCESSO

200.

39 **PATCH** **`/api/v1/castelos/:id/melhorar`** castelos:write

Melhora a muralha.

ENTRADA

- Body: `reinoId`.

REGRAS DE NEGÓCIO

- Custo: `nivelMuralha × 150` de ouro e `nivelMuralha × 80` de ferro, debitados do reino dono do território.
- Recursos insuficientes ou reino sem economia: **400**.
- A muralha sobe 1 nível e a capacidade de guardas aumenta em 50.

SUCESSO

200.

40 **POST** `/api/v1/castelos/:id/reparar` castelos:write

Restaura a integridade do castelo.

ENTRADA

- Body: `reinoId`.

REGRAS DE NEGÓCIO

- Castelo já com 100% de integridade: **400**.
- Custo fixo de 50 de ouro (**400** se insuficiente); a integridade volta a 100%.

SUCESSO

200.

Diplomacia

41 **POST** `/api/v1/diplomacia/pactos` diplomacia:write

Firma um acordo entre dois reinos.

ENTRADA

- Body: `reinoAId`, `reinoBId` (diferentes), `tipo` (ALIANCA | PACTO_NAO_AGRESSAO | ACORDO_COMERCIAL).

REGRAS DE NEGÓCIO

- Ambos os reinos precisam existir (**404**).
- Reinos em guerra direta em andamento não podem firmar acordos (**409**).
- O mesmo acordo entre os mesmos reinos não pode ser registrado duas vezes (**409**).

SUCESSO

201.

42 **GET** `/api/v1/diplomacia/pactos/:reinoId` diplomacia:read

Lista os acordos de um reino.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `status` (ATIVA | ENCERRADA), `tipo`.

SUCESSO

200.

43 **DELETE** `/api/v1/diplomacia/pactos/:id/romper` diplomacia:write

Rompe um acordo.

ENTRADA

- Body opcional: `reinoId` (quem rompe).

REGRAS DE NEGÓCIO

- Pacto já encerrado: **409**.
- Se `reinoId` for informado, ele deve fazer parte do pacto e paga multa de 50 de ouro (`DIPLOMACY_BREAK_PENALTY_OURO`); sem ouro: **422**.
- O pacto passa a ENCERRADA, com data de encerramento.

SUCESSO

200.

44 **POST** `/api/v1/diplomacia/tributos/enviar` diplomacia:write

Envia tributo de um reino para outro.

ENTRADA

- Body: `reinoOrigemId`, `reinoDestinoId` (diferentes), `tipo` (VASSALAGEM | OFERENDA_PAZ, padrão OFERENDA_PAZ), `ouro`, `alimentos`, `ferro` (≥ 0, soma > 0).

REGRAS DE NEGÓCIO

- O reino de origem precisa ter todos os recursos enviados (**422**).
- Débito na origem, crédito no destino e registro do tributo ocorrem em uma única transação serializável.

SUCESSO

201.

Tributos

45 **GET** `/api/v1/tributos` tributos:read

Lista tributos.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `tipo`, `status` (ENVIADO | ESTORNADO), `reinoOrigemId`, `reinoDestinoId`.

SUCESSO

200.

46 **GET** `/api/v1/tributos/:id` tributos:read

Detalha um tributo.

SUCESSO

200.

47 **GET** `/api/v1/tributos/balanco/:reinoId` tributos:read

Balanço de tributos do reino.

REGRAS DE NEGÓCIO

- Soma enviados e recebidos; saldo = recebidos − enviados.
- Tributos estornados não entram no balanço.

SUCESSO

200.

48 **PATCH** `/api/v1/tributos/:id/estornar` tributos:write

Estorna um tributo.

REGRAS DE NEGÓCIO

- Tributo já estornado: **409**.
- Tributos de VASSALAGEM são obrigatórios e não podem ser estornados (**422**).
- Prazo de até 24h após o envio (`TRIBUTE_REFUND_HOURS`) (**422**).
- O destino precisa ainda possuir os recursos (**422**); a devolução é feita em transação serializável.

SUCESSO

200.

Guildas, Encomendas e Membros

49 POST `/api/v1/guildas`

guildas:write

Funda uma guilda.

ENTRADA

- Body: `nome`, `oficio`, `reinoId`, `membros`? (até 50, cada um com `nome` e `especialidade`).

REGRAS DE NEGÓCIO

- Reino inexistente: **404**; nome de guilda repetido: **409**.

SUCESSO

201.

50 GET `/api/v1/guildas/:id`

guildas:read

Detalha a guilda com reino e encomendas.

SUCESSO

200.

51 GET `/api/v1/guildas/:id/membros`

guildas:read

Lista os membros da guilda.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).

SUCESSO

200.

52 POST `/api/v1/guildas/:id/encomendas`

guildas:write

Encomenda produção à guilda.

ENTRADA

- Body: `item` (ARMAS | ARMADURAS | MATERIAIS_CONSTRUCAO), `quantidade` (inteiro > 0).

REGRAS DE NEGÓCIO

- Custo por unidade: ARMAS 20 ouro + 10 ferro; ARMADURAS 30 + 25; MATERIAIS_CONSTRUCAO 10 + 5.
- O custo é debitado do reino da guilda; sem recursos: **422**.
- A encomenda nasce PENDENTE.

SUCESSO

201.

53 GET `/api/v1/encomendas`

guildas:read

Lista encomendas.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `status`, `item`, `guildaId`, `quantidadeMinima`.

SUCESSO

200.

54 GET `/api/v1/encomendas/:id`

guildas:read

Detalha uma encomenda.

SUCESSO

200.

55 PATCH `/api/v1/encomendas/:id/concluir`

guildas:write

Conclui a encomenda.

REGRAS DE NEGÓCIO

- Somente encomendas PENDENTE (**409**).
- A guilda precisa de 1 artesão a cada 10 unidades (`UNITS_PER_ARTISAN`) (**422**).

SUCESSO

200.

56 PATCH `/api/v1/encomendas/:id/cancelar`

guildas:write

Cancela a encomenda.

REGRAS DE NEGÓCIO

- Somente encomendas PENDENTE (**409**).
- Devolve 80% do ouro e do ferro ao reino (`ORDER_CANCEL_REFUND_RATE`).

SUCESSO

200 — inclui o `reembolso`.

57 GET `/api/v1/membros-guilda`

guildas:read

Lista artesãos.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `nome`, `especialidade`, `guildaId`.

SUCESSO

200.

58 POST `/api/v1/membros-guilda`

guildas:write

Adiciona um artesão.

ENTRADA

- Body: `guildaId`, `nome`, `especialidade`.

REGRAS DE NEGÓCIO

- Limite de 50 membros por guilda (`GUILD_MAX_MEMBERS`) (**422**).
- Não pode haver dois artesãos com o mesmo nome na guilda (**409**).

SUCESSO

201.

59 **PATCH** `/api/v1/membros-guilda/:id/transferir` guildas:write

Transfere o artesão de guilda.

ENTRADA

- Body: `guildaDestinoId`.

REGRAS DE NEGÓCIO

- Só entre guildas do mesmo reino e diferentes da atual (**422**).
- A guilda de destino precisa ter vaga (**422**).
- A guilda de origem não pode ficar sem artesãos suficientes para suas encomendas pendentes (**422**).

SUCESSO

200.

60 **DELETE** `/api/v1/membros-guilda/:id` guildas:write

Remove o artesão.

REGRAS DE NEGÓCIO

- Mesma regra de capacidade para encomendas pendentes (**422**).

SUCESSO

200 — membro removido.

Leis e Impostos

61 **GET** `/api/v1/leis` leis:read

Lista leis e decretos.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `titulo`, `reinoId`, `status` (ATIVA | REVOGADA).

SUCESSO

200.

62 **POST** `/api/v1/leis` leis:write

Promulga uma lei ou decreto.

ENTRADA

- Body: `titulo` (2–100), `descricao`? (até 500), `reinoId`.

REGRAS DE NEGÓCIO

- Reino inexistente: **404**.
- O título é único dentro do reino (**409**).
- A lei nasce ATIVA.

SUCESSO

201.

63 PUT `/api/v1/leis/:id/revogar`

leis:write

Revoga uma lei.

REGRAS DE NEGÓCIO

- Lei já revogada: **409**.

SUCESSO

200.

64 POST `/api/v1/leis/impostos/aplicar`

impostos:write

Aplica imposto ao reino.

ENTRADA

- Body: `reinoId`, `periodoFiscal` (ex.: 2026-Q1), `valor` (> 0), `descricao`?.

REGRAS DE NEGÓCIO

- Só pode haver um imposto por reino em cada período fiscal (**409**).

SUCESSO

201.

65 GET `/api/v1/leis/impostos/historico`

impostos:read

Histórico de impostos.

ENTRADA

- Paginação: `page` (padrão 1) e `limit` (padrão 20, máx. 100).
- Filtros: `reinoId`, `periodoFiscal`.

SUCESSO

200.